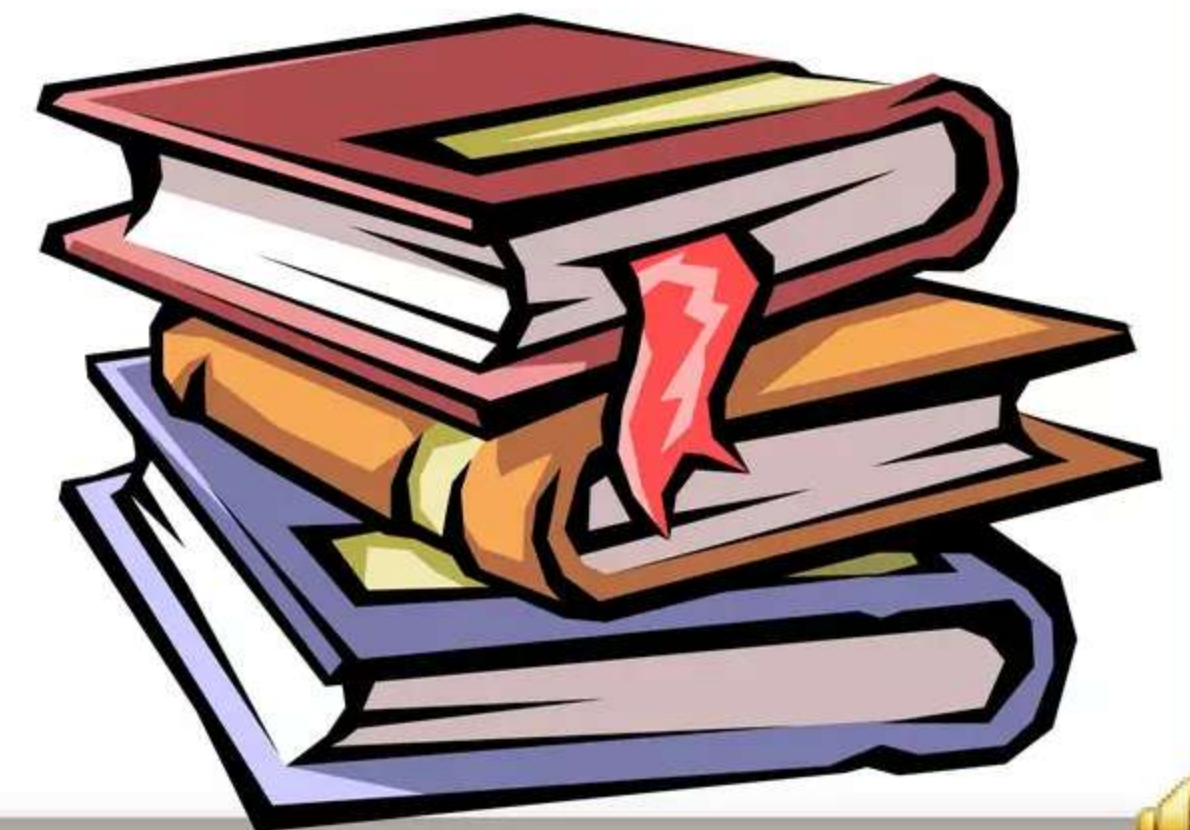


UNIT - 3

ARRAYS

Presented By

Y. N. D. ARAVIND
M.Tech, Dept of CSE
Newton's Group Of Institutions, Macherla



Session Objectives

- **Explain Arrays**
- **Explain Declaration, Initialization of Array**
- **Explain Types of Array**
- **One Dimensional, Two Dimensional and Multi Dimensional Array**

“An array is defined as fixed size sequenced collection of elements of same data type”

“An array is defined as collection of elements that share a common name”

“An array is defined as collection of homogeneous elements”

“An array is defined as collection of data items stored in contiguous memory locations”

DECLARATION OF AN ARRAY

Datatype arrayname[subscript];

Array Properties :

- 1) Arrayname should be a valid “C” variable**
- 2) Arrayname should be unique**
- 3) The elements in the array should be of same type**
- 4) Subscript (array size) cannot be negative**
- 5) Subscript must always be an integer**

TYPES OF ARRAY

One Dimensional Array's

Two Dimensional Array's

Three Dimensional Array's /

Multi Dimensional Array's

Single or One Dimensional Arrays

➤ Arrays whose elements are specified by one subscript are called One dimensional array or linear array.

➤ Syntax :

datatype arrayname[size];

If we want to represent a set of five numbers, say (45, 65, 10, 93, 50) an array variable marks. We may declare the variable marks as follows:

➤ *For Example :*

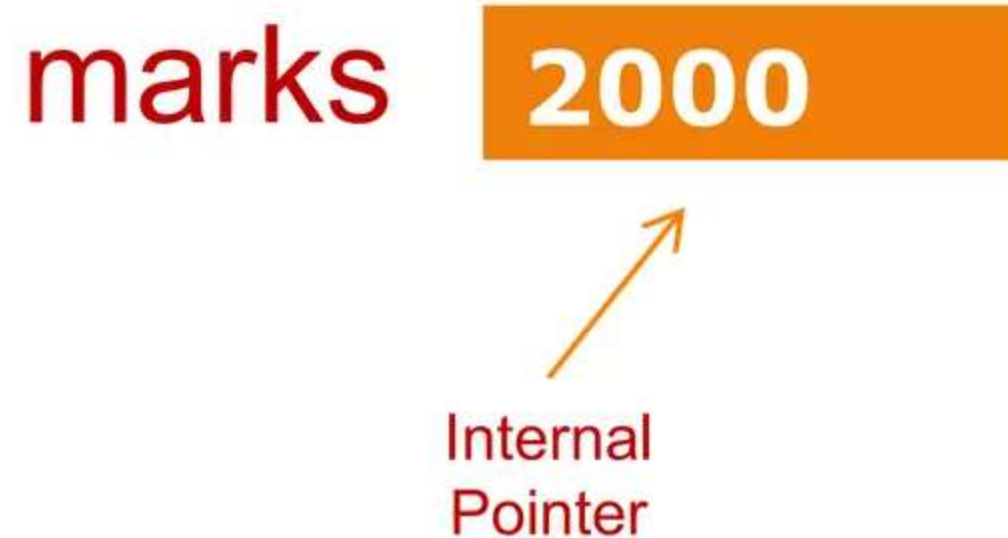
int marks[5]

➤ Note :

By default array index should starts with zero (0)

Single or One Dimensional Arrays

➤ Now the computer stores the values of the array marks after reserving five storage locations and assigning the values to array elements.



marks[0]	Garbag Value	2000
marks[1]	Garbag Value	2002
marks[2]	Garbag Value	2004
marks[3]	Garbag Value	2006
marks[4]	Garbag Value	2008

```
marks [0]=45;  
marks [1]=65;  
marks [2]=10;  
marks [3]=93;  
marks [4]=40;
```

marks[0]	45
marks[1]	65
marks[2]	10
marks[3]	93
marks[4]	40

Declaration of one dimensional Array

An array variable is declared by specifying first the base type of the array, then the name of the array variable, and then the number of elements of the array that is specified between a pair square brackets ([]). The general format of array declaration is

type variable_name[size];

Ex: int marks [100];

Here **int** specifies the data type and **marks** specifies the name of the array and **size** specifies the maximum size of the array. From the above example the maximum size of the array is 100.

Accessing elements of the Array

Once the array is declared it is possible to refer the individual elements of the array. This is done with the subscript which contains a number in the square brackets following the array name. The number specifies the element position in the array. All the array elements are numbered starting from 0. Thus **marks[2]** specifies the third element of the array.

Input and Output of Array Values

Entering data into an array/ Reading an array

```
for(i=0;i<29;i++)  
{  
    printf(" enter the marks");  
    scanf("%d",&marks[i]);  
}
```

Here the variable “i” is used in the subscript for referring various elements of the array. The for loop causes the process to be repeated for 30 times in asking to enter the students marks from user. For the first time the value of “i” is 0 read through scanf() statement and get the value of marks[0] which is the first element of the array. The process is repeated until “i” value terminates the condition.

Printing data into an array

```
for(i=0;i<29;i++)  
{  
    printf("%d", marks[i]);  
}
```

Here the variable “i” is used in the subscript for referring various elements of the array. The for loop causes the process to be repeated for 30 times in printing the students marks. For the first time the value of “i” is 0 printed through printf() statement and print the value of marks[0] which is the first element of the array. The process is repeated until “i” value terminates the condition.

Initialization of Arrays

After declaring an array the individual elements of an array are initialized otherwise they will contain garbage values. The initialization takes place in two ways.

1. Compile time initialization
2. Run time initialization

Compile time initialization

Elements of an array can be assigned initial values by following the array definition with a list of values enclosed in braces and separated by comma. The general format of initialization is

type array_name[size]={list of values};

Ex: `int marks [5] = {65, 98, 62, 48, 57};`

Defines the array marks to contain five integer elements and initializes marks [0] to 65, marks [1] to 98, marks [2] to 62, marks [3] to 48 and marks [4] to 57. If the number of initializers is less than the number of element in the array, the remaining elements are set zero.

Ex: `int m [5] = {3, 4, 8};` is equivalent to
`int m [5]= {3, 4, 8, 0, 0};`

If initializers have been provided for an array, it is not necessary to explicitly specify the array length, in which case the length is derived from the initializers.

Ex: `int m [ ] = {3, 4, 8,6};`

A character array may be initialized by a string constant, resulting in the first element of the array being set to the first character in the string, the second element to the second character, and so on. The array also receives the terminating '\0' in the string constant.

Ex: `char name [10] ="COMPUTERS";`
`char name[10] = {'c', 'o', 'm', 'p', 't', 'e', 'r', 's', '\0'};`

Initialization of Arrays

Run time initialization

An array can also be explicitly initialized at run time.

For example consider the following segment of a c program.

```
for(i=0;i<10;i++)  
{  
    scanf("%d", &x[i]);  
}
```

In the run time initialization of the arrays looping statements are almost compulsory.

Looping statements are used to initialize the values of the arrays one by one using assignment operator or through the keyboard by the user.

Write a program for entering data into an array & Reading data from an array

```
#include<stdio.h>
void main()
{
    int arr[10],i,n;
    printf("\n ENter N Elements");
    scanf("%d",&n);
    for(i=0;i<n;i++)
    {
        printf("enter arr[%d]=",i);
        scanf("%d",&arr[i]);
    }
    for(i=0;i<n;i++)
    {
        printf("%d\n",arr[i]);
    }
}
```

Input

Enter N Elements : 3
Enter arr[0] : 2
Enter arr[1] : 5
Enter arr[2] : 3

Output

2
5
3

Write a program for initialization at the time of declaration of an array & Reading data from an array

```
#include<stdio.h>
#include<conio.h>
void main()
{
    int a[5]={10,20,30,40,50};
    int i;
    clrscr();
    for(i=0;i<5;i++)
    {
        printf("%d\n",a[i]);
    }
    getch();
}
```

Output

10
20
30
40
50

Write a “C” program to sort the given number is in ascending order using one dimensional array

```
#include<stdio.h>
```

```
void main()
```

```
{  
    int i,j,n, a[10],temp;  
    printf("\n size of vector=");  
    scanf("%d",&n);  
    printf("vector elements:");  
    for (i=0;i<n;i++)  
    {  
        scanf("%d",&a[i]);  
    }  
    for(i=0;i<n-1;i++)  
        for(j=i+1;j<n;j++)  
            if(a[i]>a[j])  
            {  
                temp=a[i];  
                a[i]=a[j];  
                a[j]=temp;  
            }  
    printf("\n\nElements in asending order is=\n");  
    for(i=0;i<n;i++)  
        printf("%d",a[i]);  
    printf("\n\nElements in descending order is=\n");  
    for(i=n-1;i>=0;i--)  
        printf("%d",a[i]);  
    getch();  
}
```

Two Dimensional Arrays

If an array contains two subscripts then it is called **two dimensional arrays**. This two dimensional array is also called as **matrix**. It is called as **array of arrays**. Since it contains two subscripts it is also known as **double subscripted variable**. It is used to represent table of values containing row values and column values.

- Row → means horizontally
- Column → means vertically

Two dimensional Array Declaration

The two-dimensional array can be declared by specifying first the base type of the array, then the name of the array variable, and then the number of rows and column elements the array should be specified between a pair square brackets ([] []). Note that this value cannot be a variable and has to be an integer constant.

type variable_name[row size][column size];

Ex: *int a [3] [4]*

Two Dimensional Array Initialization

```
int ary [3][4] = {1,2,3,4,5,6,7,8,9,10,11,12}
```

The result of the above assignment will be as follows :

ary [0][0] =1	ary [0][1] =2	ary [0][2] =3	ary [0][3] =4
ary [1][0] =5	ary [1][1] =6	ary [1][2] =7	ary [1][3] =8
ary [2][0] =9	ary [2][1] =10	ary [2][2] =11	ary [2][3] =12

(OR)

1	2	3	4
5	6	7	8
9	10	11	12

```
int ary [3][4] = { {1,2,3,4}, {5,6,7,8}, {9,10,11,12} }
```

Input and Output of 2D - Array Values

Entering data into 2D array/ Reading 2D arrays

```
for(i=0;i<3;i++)  
    for(j=0;j<4;j++)  
        scanf("%d",&marks[i][j]);
```

Input

1 2 3 4 5 6 7 8 9 10 11 12

Printing data into an array

```
for(i=0;i<3;i++)  
{  
    for(j=0;j<4;j++)  
        printf("%d", marks[i][j]);  
    printf("\n");  
}
```

Output

1 2 3 4
5 6 7 8
9 10 11 12

Write a “C” program to perform the addition of two matrices

```
#include<stdio.h>
```

```
void main()
```

```
{
```

```
    int a[3][3],b[3][3],c[3][3],i,j;
```

```
    printf("Input A - Matrix\n");
```

```
    for(i=0;i<3;i++)
```

```
        for(j=0;j<3;j++)
```

```
            scanf("%d",&a[i][j]);
```

```
    printf("Input B - Matrix\n");
```

```
    for(i=0;i<3;i++)
```

```
        for(j=0;j<3;j++)
```

```
            scanf("%d",&b[i][j]);
```

```
    for(i=0;i<3;i++)
```

```
        for(j=0;j<3;j++)
```

```
            c[i][j]=a[i][j]+b[i][j];
```

```
    printf("Sum of A and B Matrix=\n");
```

```
    for(i=0;i<3;++i)
```

```
    {
```

```
        for(j=0;j<3;++j)
```

```
            printf("%d",c[i][j]);
```

```
        printf("\n");
```

```
    }
```

```
}
```

Write a “C” program to perform the subtraction of two matrices

```
#include<stdio.h>
```

```
void main()
```

```
{
```

```
    int a[3][3],b[3][3],c[3][3],i,j;
```

```
    printf("Input A - Matrix\n");
```

```
    for(i=0;i<3;i++)
```

```
        for(j=0;j<3;j++)
```

```
            scanf("%d",&a[i][j]);
```

```
    printf("Input B - Matrix\n");
```

```
    for(i=0;i<3;i++)
```

```
        for(j=0;j<3;j++)
```

```
            scanf("%d",&b[i][j]);
```

```
    for(i=0;i<3;i++)
```

```
        for(j=0;j<3;j++)
```

```
            c[i][j]=a[i][j] - b[i][j];
```

```
    printf("Sum of A and B Matrix=\n");
```

```
    for(i=0;i<3;++i)
```

```
    {
```

```
        for(j=0;j<3;++j)
```

```
            printf("%d",c[i][j]);
```

```
            printf("\n");
```

```
    }
```

```
}
```

Write a “C” program to perform matrix multiplication of two matrices

```
#include<stdio.h>
void main()
{
int a[10][10],b[10][10],c[10][10],r1,c1,r2,c2,i,j,k;
printf("Input row and column of A matrix \n");
scanf("%d %d",&r1,&c1);
printf(" Input row and column of B matrix \n");
scanf("%d %d",&r2,&c2);
/* matrix multiplication is possible only if no of
columns in first matrix is equal to no of rows in
second matrix */
if(c1!=r2)
{
    printf("Matrices cannot be multiplied \n");
}
Else
{
printf(" Matrices can be Multiplied: \n");
printf(" Input A-matrix \n");
for(i=0;i<n;++i)
for(j=0;j<m;++j)
scanf("%d",&a[i][j]);
printf(" Input B-matrix \n");
for(i=0;i<p;++i)
for(j=0;j<q;++j)
scanf("%d",&b[i][j]);
```

```
// matrix multiplication
for(i=0;i<r1;++i)
for(j=0;j<c2;++j)
{
c[i][j]=0;
for(k=0;k<c1;++k)
c[i][j]=c[i][j]+a[i][k]*b[k][j];
}
printf("Product of A and B Matrix \n");
for(i=0;i<r1;++i)
{
    for(j=0;j<c2;++j)
        printf("%d",c[i][j]);
    printf("\n");
}
getch();
}
```

Write a “C” program to find the largest and smallest numbers given in the array

```
#include<stdio.h>
```

```
void main()
```

```
{
```

```
    int i,n;
```

```
    float a[20],large,small;
```

```
    printf("\nEnter the N values=");
```

```
    scanf("%d",&n);
```

```
    printf("Enter the values one by one :\n");
```

```
    for(i=0;i<n;i++)
```

```
        scanf("%f",&a[i]);
```

```
    large=a[0];
```

```
    for(i=1;i<n;i++)
```

```
    {
```

```
        if(a[i]>large)
```

```
        large=a[i];
```

```
    }
```

```
    small=a[0];
```

```
    for(i=1;i<n;i++)
```

```
    {
```

```
        if(a[i]<small)
```

```
        small=a[i];
```

```
    }
```

```
    printf("Largest element is = %f\n",large);
```

```
    printf("Smallest element = %f\n",small);
```

```
}
```

THREE DIMENSIONAL ARRAYS / MULTIDIMENSIONAL ARRAYS

The programming language C allows three or more dimensions. But they are used rarely. Three dimensional arrays are called as array of array of arrays.

The general format of three or multidimensional arrays is

type variable_name[s1][s2][s3].....[sn]

type specifies the data type and variable_name specifies the name of the array and [s1][s2].....[sn] specifies the size of the multidimensional array.

Ex: int a[3][4][2];

Initialization of three dimensional/multidimensional arrays

The initialization of three or multi dimensional arrays is shown below.

```
int a[3][4][2]={ { {1,2},{3,4},{4,5},{6,8} },  
                 { {1,2},{3,4},{4,5},{6,8} },  
                 { {1,2},{3,4},{4,5},{6,8} } };
```

The outer array contains three elements each of which is a two dimensional array of four one dimensional arrays each of which contains two integers.

Session Summary

- ✍ **Arrayname should be a unique and valid “C” Variable name**
- ✍ **The number of elements in a multi dimensional array is the product of its subscripts**
- ✍ **Arrays can be initialized to the same type in which they are declared**
- ✍ **The character array receives the terminating ‘\0’ in the string constant**
- ✍ **The individual values in the array are called as elements**
- ✍ **It is not necessary to specify the length of an array, explicitly in case if initializers are provided for the array during declaration itself**

EXERCISES

- 1. Write a program to search an element and to find how many times it is present in the array?**
- 2. Write a program to find the sum of diagonal elements in a matrix**
- 3. Write a program to find the second largest number in an array?**
- 4. Write a program to remove the duplicate elements of the array?**
- 5. Write a program to merge two arrays and print the merged array in ascending order?**
- 6. Write a program to insert an element into an sorted array of integers?**
- 7. Write a program to display only the negative elements of the array?**

Thank You

Presented By

Y. N. D. ARAVIND
M.Tech, Dept of CSE
Newton's Group Of Institutions, Macherla

